파이썬 완전 백과사전 2권 · 딥러닝 심화: 언어와 생성

3일차

한국어 문장을 자르고, 읽고, 분류하기

11~15차시 · 책 13장 한국어 NLP 실전

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/13-korean-nlp

오늘의 시간표

차시	주제	책 범위	오래 걸리는 예제
11	영화 리뷰, 한국어가 어려운 이유, 글자·어절	13장 들어가며 ~ 13.3	b01 첫 실행 10초
12	형태소, 서브워드, 사전학습 토크나이저	13.4 ~ 13.6	b12 13초
13	임베딩, 처음부터 학습하는 기준 모델	13.7 ~ 13.8	b15 18초 · b18 19초
14	KLUE BERT 미세조정, 평가, datasets	13.9 ~ 13.11	b22 학습 113초 (GPU)
15	실습: 뉴스 제목 주제 분류기, 정리	13.12 ~ 13.13	b26 학습 98초 (GPU)

먼저 받아 둘 것: klue/bert-base 가중치 **445MB** · NSMC **19MB** · KLUE YNAT **5MB**

오래 걸리는 학습은 **차시 첫머리에 실행**을 걸어 두고 설명한다

준비: 13-korean-nlp 폴더에서 실행 · 모델·데이터는 첫 실행 때 받아 ~/.cache/huggingface에 보관 · 로그인 필요 없음

11차시

11

한국어가 어려운 이유를 숫자로 보기

책 13장 들어가며 ~ 13.3절

학습 목표

- 네이버 영화 리뷰(NSMC)를 받아 읽고 건강검진한다
- 교착어, 들쭉날쭉한 띄어쓰기, 유니코드 문제를 데이터로 확인한다
- 글자와 어절로 잘라 사전 크기와 문장 길이의 줄다리기를 본다

13장 "아버지가방에들어가신다"를 어디서 끊을까

```
문장 = "아버지가방에들어가신다"
print("띄어쓰기로:", 문장.split())
print("글자로    :", list(문장))
print("형태소로  :", [t.form for t in Kiwi().tokenize(문장, compatible_jamo=True)])
print("KLUE BERT :", AutoTokenizer.from_pretrained("klue/bert-base").tokenize(문장))
```

```
띄어쓰기로: ['아버지가방에들어가신다']
글자로    : ['아', '버', '지', '가', '방', '에', '들', '어', '가', '신', '다']
형태소로  : ['아버지', '가', '방', '에', '들어가', '시', 'ㄴ다']
KLUE BERT : ['아버지', '##가방', '##에', '##들', '##어가', '##신', '##다']
```

형태소 분석기는 사람처럼 바르게, KLUE BERT는 **가방**으로 끊었다. 그래도 모델은 문맥으로 메운다

13.1 받아서 읽는 도구: 한국어도구.py

```
def NSMC_읽기(파일: str, 개수: int | None = None, 씨앗: int = 0) -> list[tuple[str, int]]:
    """(문장, 라벨) 목록을 돌려준다. 빈 문장은 버리고, 개수를 주면 무작위로 그만큼 고른다."""
    NSMC_받기()                                              # 없을 때만 내려받는다 (약 19MB)
    with open(폴더 / 파일, encoding="utf-8", newline="") as f:
        줄들 = list(csv.reader(f, delimiter="\t"))[1:]       # 첫 줄은 열 이름
    자료 = [(문장.strip(), int(라벨)) for _, 문장, 라벨 in 줄들 if 문장.strip()]
    if 개수 is not None:
        자료 = random.Random(씨앗).sample(자료, 개수)
    return 자료
```

split 대신 csv

큰따옴표가 든 리뷰를 따옴표로 감싸고 안쪽을 두 번 겹쳐 적었다. csv가 원래 문장으로 풀어 준다

나만 쓰는 난수 생성기

random.Random(씨앗)으로 고르면 언제나 같은 리뷰들. 예제끼리 숫자를 비교할 수 있다

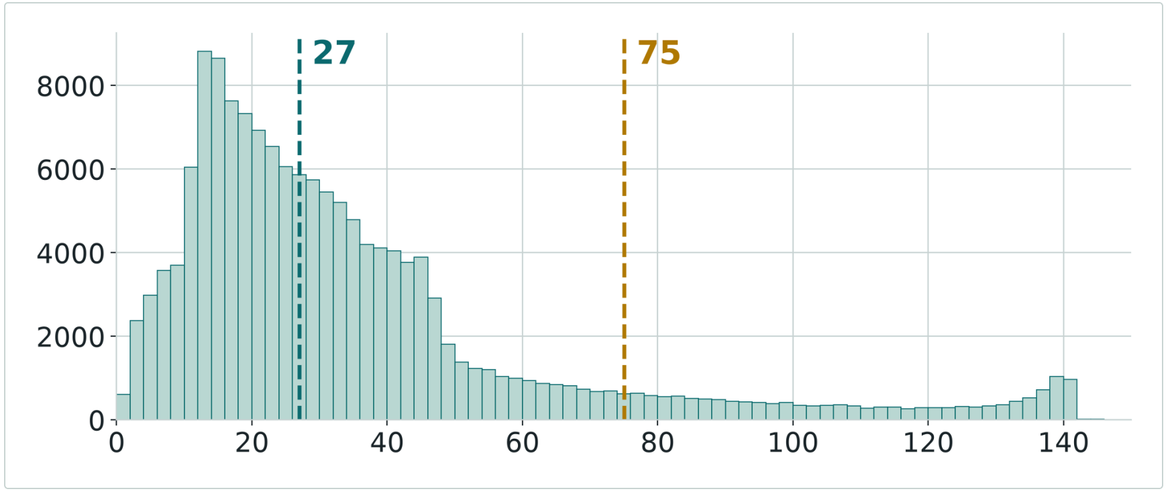
11차시 · 13.1 이 장의 데이터: 네이버 영화 리뷰 · 예제 13-korean-nlp/한국어도구.py (핵심 부분)

13.1 둘러보기: 건강검진부터

```
학습 149,995개, 시험 49,997개
긍정(1) 비율: 학습 0.499, 시험 0.503
띄어쓰기가 하나도 없음: 8.3%
'ㅋ'나 'ㅎ'가 들어 있음: 8.1%
마침표를 두 개 이상 이어 씀: 28.3%
영어 알파벳이 들어 있음: 5.9%
```

긍정이 반쯤이라 **불균형 걱정 없음**. 교과서 문장이 아니라 사람들이 실제로 쓰는 말

리뷰 글자 수 분포 · 중앙값 27 · 90% 지점 75 · 가장 긴 것 146



가로는 글자 수, 세로는 리뷰 수

13.2 말끝이 끝없이 바뀐다: 교착어

재밌다, 재밌어요, 재밌네요, 재밌었다, 재밌고… 뜻의 중심에 조사·어미가 붙는다. 띄어쓰기로 자른 덩어리 = **어절**

357,862
서로 다른 어절 (리뷰 15만 개)

79.6%
딱 한 번만 나온 어절

1,778
'재밌'으로 시작하는 어절의 가짓수

25.2%
시험 어절 중 사전에 없는 것 (OOV)

자주 나온 것: 재밌게(1343), 재밌다(375), 재밌는(374), 재밌고(339), 재밌어요(271), 재밌음(238), **재밌다.(193)**, 재밌는데(182)

문장부호 하나만 붙어도 다른 어절. 시험 문장의 네 단어 중 하나가 **처음 보는 외국어**인 셈

13.2 띄어쓰기가 들쭉날쭉하다

붙여 씀	띄어 씀
볼만한 **406**번	볼 만한 **28**번
재미없다 **997**번	재미 없다 **65**번
시간가는줄 **109**번	시간 가는 줄 **27**번

둘 다 맞는 표기, 뜻도 같다. 그런데 띄어쓰기로 자르면 **전혀 다른 토큰**

15글자 이상인데 띄어쓰기가 하나도 없는 리뷰 **2,336개**

Kiwi의 space()로 띄어쓰기를 다시 넣으면 (예제용으로 지은 문장)

어제동생이랑보러갔는데생각보다훨씬재밌었어요
→ **어 제** 동생이랑 보러 갔는데 생각보다 훨씬 재밌었어요
배우들연기는좋은데결말이너무허무해서아쉬움
→ 배우들 연기는 **좋은 데** 결말이 너무 허무해서 아쉬움

13.2 눈에는 같은데 다른 글자

```
완성형 = "한글"
분해형 = unicodedata.normalize("NFD", 완성형)     # 초성·중성·종성으로 쪼갠 형태
print(완성형 == 분해형, len(완성형), len(분해형))
print("분해형이 '글'로 끝나나?", 분해형.endswith("글"))
print(unicodedata.normalize("NFC", 분해형) == 완성형)   # 다시 합치면 같아진다

def 정규화(문장: str) -> str:
    """처리하기 전에 언제나 완성형(NFC)으로 맞춘다."""
    return unicodedata.normalize("NFC", 문장)
```

```
False 2 6
분해형이 '글'로 끝나나? False
True
```

맥의 파일 이름, 압축 파일, 일부 웹 글이 분해형이다. 맨 처음에 **NFC로 맞추는 한 줄**을 넣어 둔다

13.3 사전 크기와 문장 길이의 줄다리기

레고로 치면 1×1 블록만 있으면 무엇이든 만들지만 많이 끼워야 하고, 집 모양 블록만 있으면 한 번에 끝나지만 종류가 끝도 없다

```
학습 = [문장 for 문장, _ in NSMC_읽기("ratings_train.txt", 20000)]   # 리뷰 2만 개로 사전을 만들고
시험 = [문장 for 문장, _ in NSMC_읽기("ratings_test.txt", 5000)]     # 시험 리뷰 5천 개를 자른다
비교("글자", lambda s: list(s.replace(" ", "")))
비교("어절", str.split)
```

자르기	사전	문장당 토큰	모르는 토큰
글자	1,942개	**29.0**	0.1% 길다
어절	69,651개	7.7	**38.3%** 모른다

11차시 · 13.3 토큰화 ①: 글자와 어절 · 예제 13-korean-nlp/b07_char_word.py (핵심 부분, 출력을 표로)

직접 실행 10분

1. b03_nsmc_look.py 끝에 print(학습[:5])를 붙여 원문 다섯 개를 읽는다. 띄어쓰기·ㅋㅋ·마침표를 찾아본다
2. b04를 복사해 "재밌"을 **"재미없"**으로 바꾼다. 몇 가지 어절이 나올까?
3. b07을 복사해 학습 리뷰를 2만 개에서 **전체(15만 개)**로 늘린다. 어절의 모르는 토큰 38.3%는 얼마로 줄까?

```
재미없_계열 = sorted((a for a in 어절_수 if a.startswith("재미없")), key=lambda a: -어절_수[a])
학습 = [문장 for 문장, _ in NSMC_읽기("ratings_train.txt")]          # 3번: 개수를 빼면 전부
```

어절 사전으로 시험 리뷰를 읽을 때 OOV가 많은 이유가 아닌 것은?

Q

가 조사와 어미가 붙어 모양이 계속 바뀐다

나 문장부호 하나만 붙어도 다른 어절이 된다

다 사람마다 띄어쓰기가 다르다

라 리뷰에 쓰인 글자 종류가 2천 개 남짓이다

11차시 정리

NSMC 리뷰 15만 개는 긍정·부정이 반반, 한 줄 평이 대부분이고 **사람이 실제로 쓰는 말**이다

교착어라 어절이 폭발한다: 35만 가지, 79.6%가 한 번만, 시험 어절 넷 중 하나가 OOV

글자는 사전이 작고 문장이 길다, 어절은 그 반대에 OOV가 심하다. 처리 전에 NFC로 맞춘다

다음 차시: 줄다리기의 좋은 자리 · 형태소 분석과 서브워드, 사전학습 토크나이저

12차시

12

형태소, 서브워드, 사전학습 토크나이저

책 13.4 ~ 13.6절

학습 목표

- Kiwi로 형태소와 품사 태그를 얻고 장단점을 말한다
- BPE가 자주 붙는 두 조각을 합쳐 사전을 만드는 과정을 손으로 따라간다
- KLUE BERT 토크나이저의 input_ids, attention_mask, 특수 토큰을 읽는다

13.4 뜻의 최소 단위로 자르기: Kiwi

```
결과 = kiwi.tokenize(문장, compatible_jamo=True)
print(" ".join(f"{t.form}/{t.tag}" for t in 결과))
```

```
이/MM 영화/NNG 진짜/MAG 재밌/VA 었/EP 어요/EF ㅋㅋ/SW
재미없/VA 지/EC 는/JX 않/VX 었/EP 다/EF
스토리/NNG 가/JKS 산/NNG 으로/JKB 가/VV ㅁ/EF
존/NNP 잼/NNG 노/NNG 잼/NNG 꿀/NNG 잼/NNG
['배우', '연기', '어색', '몰입', '되']
```

VA 형용사 · EP 과거(었) · EF 끝맺는 어미 · JKS 주격 조사 · SW 기호

이중 부정의 뼈대는 그대로, 그러나 "존잼"은 **존(고유 명사) / 잼**

13.4 사전이 얼마나 줄어들까

자르기	사전	문장당 토큰	모르는 토큰	
글자	1,942개	29.0	0.1%	
어절	69,651개	7.7	38.3%	
형태소	**18,793개**	**20.0**	**2.6%**	사이에 놓였다

한쪽 리뷰에서 유난히 많이 나온 형용사

긍정

멋지, 깊, 재미있, **아프**, **슬프**, 멋있

부정

재미없, 심하, 어설프, 어이없, 뻔하, 지루하

영화 리뷰에서 "슬프다"는 칭찬. 감성은 단어 하나가 아니라 **분야와 문맥**에 달렸다

13.5 자주 붙어 다니는 글자를 묶기: BPE

```
어절_수 = {"재밌다": 5, "재밌어요": 6, "재밌네요": 3, "재미없다": 4, "재미없어요": 3, "좋네요": 4}
쪼갬 = {어절: list(어절) for 어절 in 어절_수}          # 처음에는 모두 글자 단위
for 단계 in range(1, 7):
    짝, 횟수 = 짝_세기().most_common(1)[0]        # 가장 자주 붙어 나온 두 조각
    합치기(짝)                                     # 모든 어절에서 한 조각으로
```

```
1단계: 재 + 밌 (14번)
2단계: 어 + 요 (9번)
3단계: 네 + 요 (7번)
4단계: 재 + 미 (7번)
5단계: 재미 + 없 (7번)
6단계: 재밌 + 어요 (6번)
```

```
재밌다 → ['재밌', '다']
재밌어요 → ['재밌어요']
재밌네요 → ['재밌', '네요']
재미없다 → ['재미없', '다']
재미없어요 → ['재미없', '어요']
좋네요 → ['좋', '네요']
```

문법을 가르치지 않았는데 뼈대와 어미가 조각이 됐다. 자주 쓰는 말은 **통째로**, 드문 말은 **잘게**

13.5 tokenizers로 진짜 BPE 학습하기

```
for 크기 in [500, 4000, 32000]:
    토크나이저 = Tokenizer(models.BPE(unk_token="[UNK]"))
    토크나이저.pre_tokenizer = pre_tokenizers.Whitespace()       # 먼저 공백·문장부호로 자른다
    훈련 = trainers.BpeTrainer(vocab_size=크기, special_tokens=["[PAD]", "[UNK]"],
                               show_progress=False)
    토크나이저.train_from_iterator(학습, 훈련)                    # 리뷰 15만 개
```

```
사전  3,005개: 문장당 29.0토큰 | 시 간 가 는 줄 모 르 고 봤 네 요 끝 까 지 재 밌 었 어 요
사전  4,000개: 문장당 19.7토큰 | 시간 가는 줄 모르 고 봤 네요 끝까지 재밌 었어요
사전 32,000개: 문장당 13.1토큰 | 시간 가는 줄 모르고 봤네요 끝까지 재밌었어요
```

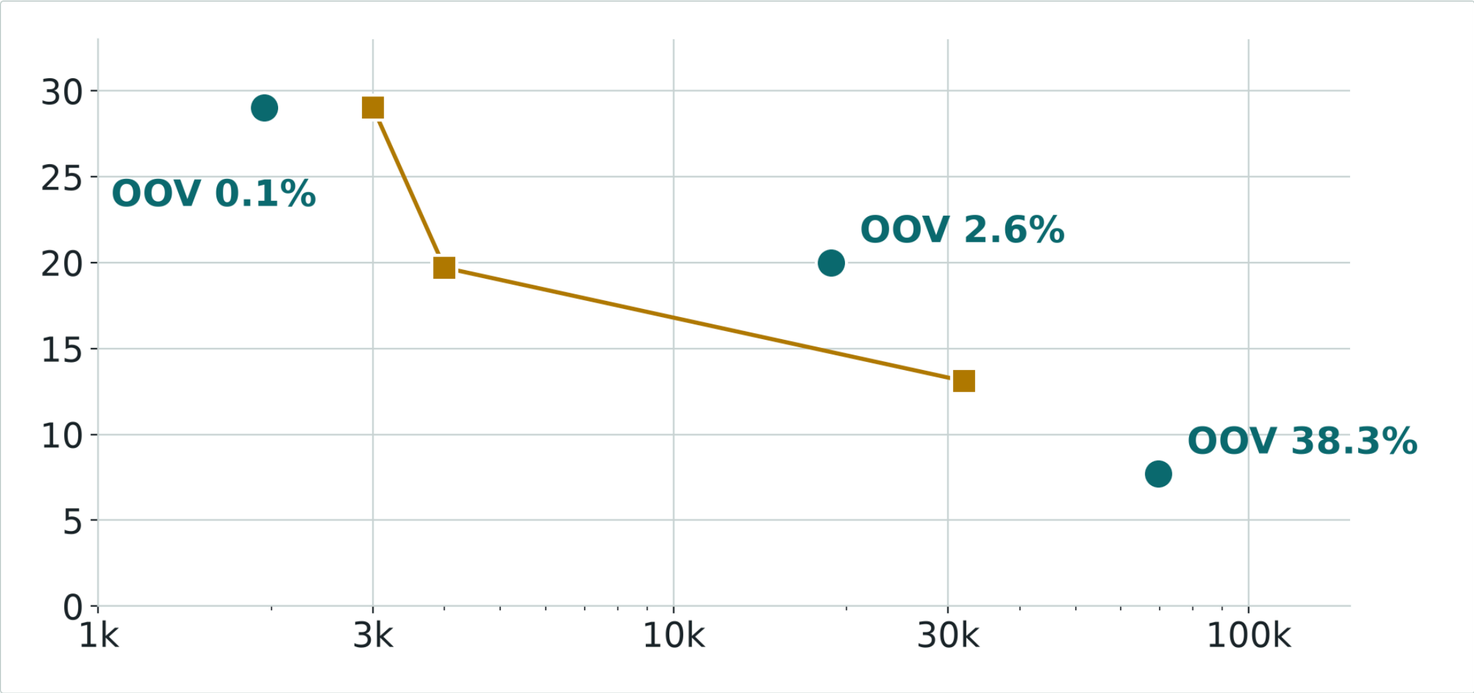
500을 요청했는데 **3,005개**: 글자 종류만 3천 개가 넘는다

4,000개: 형태소(20.0)와 거의 같은 길이를 분석기 없이

32,000개: 짧아지지만 토큰마다 드물어 배우기 어렵다

13.3~13.5 줄다리기를 한 장에

● 글자 · 형태소 · 어절 (왼쪽부터) · ■ BPE 3,005 · 4,000 · 32,000



가로는 사전 크기(로그 눈금), 세로는 시험 문장당 토큰 수

BPE 4,000개는 형태소의 **5분의 1쯤 되는 사전**으로 같은 길이

어절은 사전이 크고 문장이 짧지만 **넷에 하나 이상 모른다**

BPE는 글자까지 쪼갤 수 있어 OOV가 거의 없다

12차시 · 13.3~13.5 토큰화 정리 · b07, b09, b11 출력 숫자를 한 그림에 (글자·형태소·어절은 학습 2만, BPE는 학습 15만, 시험 5천)

13.5 BPE의 친척: WordPiece와 SentencePiece

방식	합칠 짝 고르는 기준	표시	쓰는 모델 예
BPE	가장 자주 붙어 나오는 짝	따로 없음	GPT 계열
WordPiece	합쳤을 때 데이터의 확률을 가장 많이 올리는 짝	중간 조각 앞에 ##	BERT, KLUE BERT
SentencePiece (Unigram)	큰 사전에서 시작해 덜 중요한 조각을 지워 나감	첫 조각 앞에 ▁	XLM-R, T5

"아버지 / ##가방 / ##에": ##는 "앞 조각에 붙어 있던 조각". 다시 문장으로 붙일 때 어디를 띄울지 안다

13.6 세 모델의 토크나이저 비교

```
예문 = "이 영화 진짜 재밌었어요 배우들 연기도 최고!"
토크나이저 = AutoTokenizer.from_pretrained(이름)        # 처음 한 번만 내려받고, 다음부터는 캐시에서
print(" ", 토크나이저.tokenize(예문))
```

```
klue/bert-base (사전 32,000개)
  문장당 20.3토큰, [UNK] 0.32%
  ['이', '영화', '진짜', '재밌', '##었', '##어요', '배우', '##들', '연기', '##도', '최고', '!']
google-bert/bert-base-multilingual-cased (사전 119,547개)
  문장당 24.1토큰, [UNK] 1.77%
  ['이', '영화', '진', '##짜', '[UNK]', '배우', '##들', '연', '##기도', '최고', '!']
FacebookAI/xlm-roberta-base (사전 250,002개)
  문장당 21.9토큰, [UNK] 0.11%
  ['▁이', '▁영화', '▁진짜', '▁재', '밌', '었어요', '▁배우', '들', '▁연기', '도', '▁최고', '!']
```

사전이 가장 작은 KLUE BERT가 문장이 **가장 짧다**. 다국어 BERT는 "재밌었어요"를 통째로 잃었다

12차시 · 13.6 사전학습 토크나이저 쓰기 · 예제 13-korean-nlp/b12_hf_compare.py (핵심 부분)

13.6 토크나이저가 돌려주는 것

```
인코딩 = 토크나이저("재밌었어요")
print(인코딩)
print(토크나이저.convert_ids_to_tokens(인코딩["input_ids"]))
print(토크나이저.decode(인코딩["input_ids"]))
```

```
{'input_ids': [2, 7478, 2359, 10283, 3], 'token_type_ids': [0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1]}
['[CLS]', '재밌', '##었', '##어요', '[SEP]']
[CLS] 재밌었어요 [SEP]
```

input_ids

토큰 번호. 앞뒤에 특수 토큰 [CLS] 2번, [SEP] 3번

token_type_ids

두 문장을 넣을 때 첫 문장 0, 둘째 1

attention_mask

진짜 토큰 1, 패딩 0

13.6 여러 문장을 한꺼번에: 패딩과 자르기

```
배치 = 토크나이저(["최고", "배우들 연기가 너무 어색해서 몰입이 안 됨", "두 시간이 순식간에 지나갔다"],
                  padding=True, truncation=True, max_length=10, return_tensors="pt")
```

```
tensor([[    2,  3841,     3,     0,     0,     0,     0,     0,     0,     0],
        [    2,  4165,  2031,  4483,  2116,  3760,  9977,  6396, 11306,     3],
        [    2,   864,  3641,  2052,  9390,  2170, 12613,  2062,     3,     0]])
tensor([[1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 0]])
['[CLS]', '배우', '##들', '연기', '##가', '너무', '어색', '##해서', '몰입', '[SEP]']
```

기초 과정의 collate_fn이 하던 일을 한 줄로. 10토큰에서 잘려도 **[SEP] 자리는 남긴다**

직접 실행 10분

1. b10을 복사해 합치기를 **9단계**까지 돌린다. 여섯 어절은 마지막에 어떻게 잘릴까?
2. b11을 복사해 사전 크기를 **8,000과 16,000**으로 바꾼다. 문장당 토큰 수를 표에 더한다
3. b08을 복사해 신조어가 든 문장을 넣어 본다: "갓띵작 인정합니다", "스포 당해서 김샜음"

```
for 단계 in range(1, 10):                                  # 1번
for 크기 in [8000, 16000]:                                 # 2번: 끝의 save 두 줄은 지운다
```

12차시 · 직접 실행 · 1번·3번은 1초 안쪽, 2번은 4초 (13-korean-nlp 폴더에서 실행)

KLUE BERT 모델에 넣을 문장을 자를 때 써야 하는 것은?

Q

가 Kiwi 형태소 분석 결과

나 리뷰 15만 개로 새로 학습한 BPE

다 AutoTokenizer.from_pretrained("klue/bert-base")

라 사전이 가장 큰 XLM-R 토크나이저

12차시 정리

형태소 분석(Kiwi)은 뜻 단위로 바르게 자르지만 신조어에 약하다

서브워드(BPE, WordPiece, SentencePiece)는 자주 붙는 조각을 묶어 OOV 없이 중간을 찾는다

사전학습 토크나이저는 모델과 한 몸: **input_ids, attention_mask,** [CLS]·[SEP]·[PAD]

다음 차시: 토큰 번호를 화살표로 · 임베딩과 처음부터 학습하는 기준 모델

13차시

13

임베딩, 그리고 처음부터 학습하는 기준 모델

책 13.7 ~ 13.8절

학습 목표

- 임베딩이 번호로 줄을 꺼내는 표라는 것을 설명한다
- 코사인 유사도로 단어·문장을 비교하고, 문맥 임베딩을 확인한다
- 평균 임베딩 분류기로 비교의 기준 점수를 잰다

13.7 임베딩: 번호표로 꺼내는 표

원-핫이라면

토큰 하나가 **3만 2천 칸**. 너무 크고, 모든 토큰 사이 거리가 똑같아 "재밌"과 "흥미"가 비슷하다는 걸 담지 못한다

임베딩

토큰마다 **작은 화살표**(숫자 몇십~몇백 개) 한 줄. 원-핫 @ 표와 같은 결과를, 곱셈 없이 그 번호의 줄만 꺼내서

padding_idx=0

0번 줄은 0으로 채워지고 학습해도 그대로. 채워 넣은 칸은 **아무 뜻도 갖지 않는다**

```
nn.Embedding(사전 크기, 칸 수, padding_idx=0)     # 처음엔 무작위, 학습하면서 고쳐진다
```

13.7 KLUE BERT의 임베딩 표 들여다보기

```
표 = 모델.get_input_embeddings().weight.detach()      # (사전 크기, 768): 토큰마다 화살표 하나
단위 = 표 / 표.norm(dim=1, keepdim=True)              # 길이를 1로 맞추고
for 단어 in ["영화", "최고", "지루", "배우", "서울"]:
    번호 = 토크나이저.convert_tokens_to_ids(단어)
    유사도 = 단위 @ 단위[번호]                         # 모든 토큰과의 코사인 유사도를 한 번에
    상위 = 유사도.topk(6).indices[1:]                  # 1등은 자기 자신이라 뺀다
```

```
(32000, 768) 93.8 MB
영화 → ['드라마', '##영화', '소설', '연극', '영상']
최고 → ['최대', '최상', '가장', '최악', '최저']
지루 → ['따분', '식상', '지겹', '밋밋', '재미없']
배우 → ['##배우', '배울', '배웠', '배운다', '연기자']
서울 → ['부산', '인천', '서울시', '한국', '대구']
```

아무도 알려 주지 않았는데 **비슷한 자리에 쓰이는 말**끼리 모였다

13.7 가깝다는 것의 두 가지 함정

최고 → 최악

"최고의 연기", "최악의 연기"처럼 **같은 자리**에 쓰이니 화살표가 가깝다

가깝다 = 쓰이는 자리가 비슷하다. 뜻이 같다는 뜻이 아니다. 감성 분류에는 치명적

배우 → 배울, 배웠

연기하는 배우와 공부하는 배우다가 **같은 행 하나**를 나눠 쓴다

임베딩 표만으로는 둘을 구분할 수 없다

해결은 둘: 문맥이 뜻을 정한다(다음 장) · 감성은 미세조정으로 배운다(14차시)

13.7 문맥이 뜻을 정한다: 문맥 임베딩

"배"가 든 문장 여섯

1. 배가 고파서 밥을 먹었다
2. 점심을 굶었더니 배가 아프다
3. 배를 타고 섬에 갔다
4. 항구에 큰 배가 들어왔다
5. 깎아 놓은 배가 달고 시원하다
6. 사과와 배를 한 상자씩 샀다

1·2 몸, 3·4 탈것, 5·6 과일

인코더를 거친 "배"끼리의 코사인 유사도

	1	2	3	4	5	6
1	1.	**0.89**	0.69	0.65	0.64	0.65
2	**0.89**	1.	0.61	0.62	0.67	0.65
3	0.69	0.61	1.	**0.82**	0.58	0.66
4	0.65	0.62	**0.82**	1.	0.6	0.64
5	0.64	0.67	0.58	0.6	1.	**0.71**
6	0.65	0.65	0.66	0.64	**0.71**	1.

같은 뜻끼리 **0.89** · **0.82** · **0.71**, 다른 뜻끼리 0.58~0.69. 입력 임베딩이었다면 모두 1

13.7 문장 하나를 화살표 하나로

```
출력 = 모델(**입력).last_hidden_state                    # (문장, 토큰, 768)
마스크 = 입력["attention_mask"].unsqueeze(-1).float()      # (문장, 토큰, 1)
평균 = (출력 * 마스크).sum(dim=1) / 마스크.sum(dim=1)        # 패딩 칸은 빼고 평균
```

문장	1	2	3	4
1 연기도 좋고 스토리도 탄탄하다	1.	0.86	**0.8**	0.47
2 배우들 연기가 훌륭하고 줄거리도 짜임새 있다	0.86	1.	0.74	0.45
3 연기도 별로고 스토리도 엉성하다	**0.8**	0.74	1.	0.46
4 오늘 저녁 메뉴는 김치찌개다	0.47	0.45	0.46	1.

무엇에 대한 문장인지는 잘 잡는다

칭찬과 깎아내림이 0.80: **좋다·나쁘다**는 크게 신경 쓰지 않는다

13.8 기준 모델: 평균 임베딩 분류기

비싼 모델이 얼마나 좋아졌는지 말하려면 먼저 **기준(baseline)**이 있어야 한다

```
class 평균_임베딩_분류기(nn.Module):
    """토큰 화살표들의 평균 하나로 긍정/부정을 가린다. 순서는 보지 않는다."""
    def __init__(self, 사전_크기: int, 칸수: int = 64) -> None:
        super().__init__()
        self.임베딩 = nn.Embedding(사전_크기, 칸수, padding_idx=0)
        self.출력 = nn.Linear(칸수, 2)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        마스크 = (x != 0).unsqueeze(-1).float()                    # 패딩(0번)은 평균에서 뺀다
        평균 = (self.임베딩(x) * 마스크).sum(1) / 마스크.sum(1).clamp(min=1)
        return self.출력(평균)
```

BPE 사전 8,000개(리뷰 15만으로 학습) · 64칸 · Adam lr 3e-3 · 3에폭 · CPU로 충분

13.8 순서를 안 봐도 85%



막대 길이 = 시험 정확도 (0~100%, 같은 시험 리뷰 5천 개)

데이터가 늘수록 좋아지지만 **늘어나는 폭은 줄어든다**

"재미없지는 않았다"와 "않았다 재미없지는"이 **같은 평균**

13.8 학습된 임베딩이 가리키는 긍정 쪽

```
방향 = 모델.출력.weight[1] - 모델.출력.weight[0]
점수 = (모델.임베딩.weight @ 방향).detach()
```

```
# 긍정 가중치 - 부정 가중치
  # 토큰마다 그 방향으로 얼마나 뻗었나
```

긍정 쪽 끝

울컥, 돌아보게, 탄탄하고, 좋았어요, 질리지, 여운이, 좋았고, **저평가**

부정 쪽 끝

최악의, 최악, 최악이다, **높아서**, 엉성하고, 시간낭비, 개판, 아깝고

사전학습 임베딩은 최고와 최악을 이웃으로, 감성용 임베딩은 최악을 **부정 맨 끝**에. 무엇을 위해 배웠느냐가 화살표를 정한다

직접 실행 10분

1. b15를 복사해 이웃을 찾을 단어를 바꾼다: **감동, 공포, 연기, 재밌, 별로**. "최고 → 최악" 같은 함정이 또 있나?
2. b17을 복사해 다섯째 문장 **"배우들 연기가 형편없고 줄거리도 엉망이다"**를 더한다. 어느 문장과 가장 가까울까?
3. b18을 복사해 칸수를 64에서 **8**로 줄이고 15만 개 학습만 남긴다. 85.00%는 얼마가 될까?

```
def __init__(self, 사전_크기: int, 칸수: int = 8) -> None:      # 3번
# 학습하고_평가(전체[:5000]), 학습하고_평가(전체[:20000]) 두 줄은 지운다
```

13차시 · 직접 실행 · 1번 6초, 2번 5초, 3번 21초 (KLUE BERT 가중치를 받아 둔 뒤 이 컴퓨터에서 잰 값)

KLUE BERT 임베딩 표에서 "최고"의 이웃에 "최악"이 있는 까닭은?

Q

가 두 단어의 뜻이 같아서

나 문장에서 쓰이는 자리가 비슷해서

다 토큰 번호가 서로 가까워서

라 패딩 칸이 평균에 섞여서

13차시 정리

임베딩은 번호로 줄을 꺼내는 표. 코사인 유사도로 이웃을 찾으면 쓰이는 자리가 비슷한 말이 모인다

인코더를 거친 **문맥 임베딩**은 같은 토큰도 문맥에 따라 다르다. 평균 내면 문장 화살표

처음부터 학습한 **기준 모델**: 5천 개 74.16%, 15만 개 85.00%

다음 차시: KLUE BERT를 리뷰 5천 개로 미세조정해 85.00%와 겨룬다

14차시

14

사전학습 인코더 미세조정과 평가

책 13.9 ~ 13.11절

학습 목표

- 라이선스와 크기를 확인하고 KLUE BERT를 미세조정한다
- 혼동 행렬·F1과 자신 있게 틀린 예제로 평가한다
- pipeline으로 추론하고 datasets로 데이터를 다룬다

지금 실행: **uv run python b22_finetune.py** · GPU 113초

13.9 빌려 쓰기 전에: 라이선스와 크기

```
klue/bert-base: 라이선스 cc-by-sa-4.0, 로그인 필요 False, 파라미터 111,243,522개
google-bert/bert-base-multilingual-cased: 라이선스 apache-2.0, 로그인 필요 False, 파라미터 178,566,653개
FacebookAI/xlm-roberta-base: 라이선스 mit, 로그인 필요 False, 파라미터 278,885,778개
```

```
  bert.embeddings   24,972,288개
  bert.encoder      85,054,464개
  bert.pooler          590,592개
  classifier             1,538개
인코더 층 수: 12
전체 110,618,882개, float32로 422 MB
학습 중 최소 1.65 GB (중간 계산값은 별도)
```

CC BY-SA 4.0: 상업적 이용 가능, 출처 표시, 고쳐 배포하면 같은 라이선스로

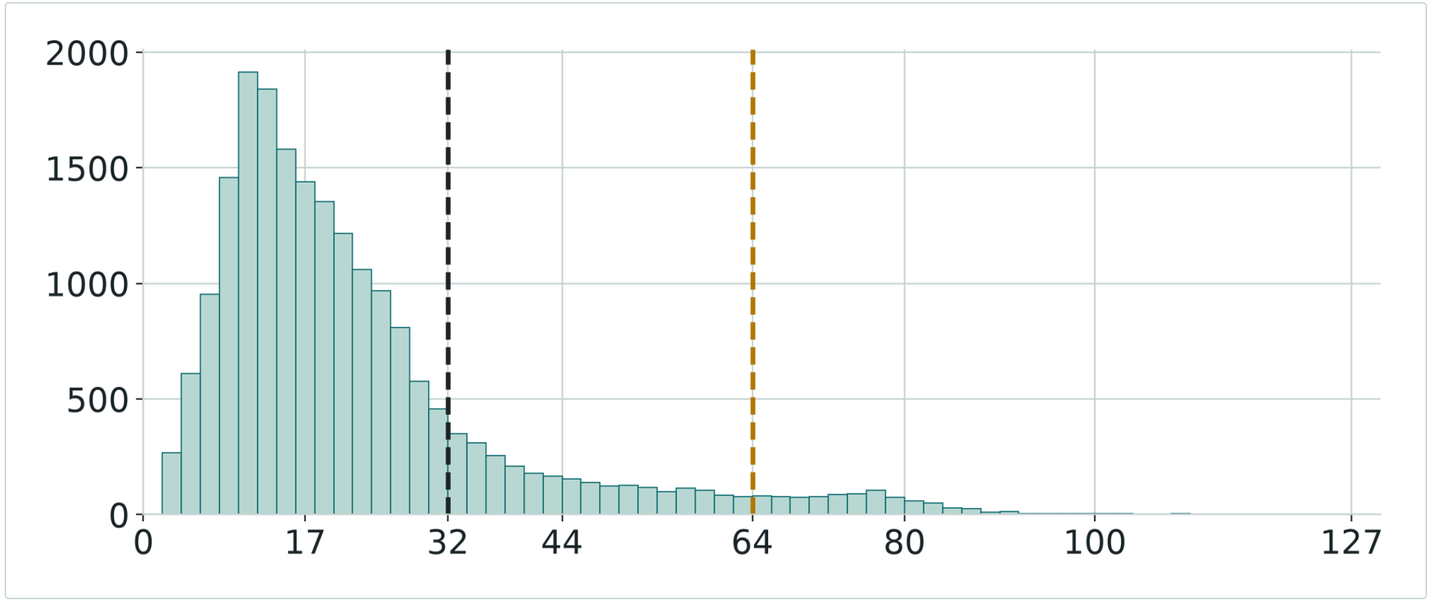
새로 만든 건 **nn.Linear(768, 2)** 1,538개뿐, 나머지 1억 1천만 개는 빌려 온 것

AdamW 학습 = 파라미터 + 기울기 + 상태 둘 = **4배**

14차시 · 13.9 라이선스와 모델 크기 · 예제 13-korean-nlp/b19_license.py, b20_model_size.py (출력 일부)

13.9 몇 토큰에서 자를까

리뷰 2만 개의 KLUE BERT 토큰 수 · -- max_length 32 · -- max_length 64



가로는 토큰 수([CLS]·[SEP] 포함), 세로는 리뷰 수

지점	토큰
50%	17
90%	44
99%	80
가장 긴 것	127

max_length	잘리는 리뷰
32	16.6%
64	**4.2%**
128	0.0%

14차시 · 13.9 몇 토큰에서 자를까요? · 예제 13-korean-nlp/b21_maxlen.py 출력과, 같은 데이터로 그린 토큰 수 분포

13.9 미세조정하기: 학습 루프는 그대로

```
모델 = AutoModelForSequenceClassification.from_pretrained(
    이름, num_labels=2, id2label={0: "부정", 1: "긍정"}, label2id={"부정": 0, "긍정": 1}).to(장치)
def 묶기(배치: list[tuple[str, int]]) -> dict[str, torch.Tensor]:
    입력 = 토크나이저([s for s, _ in 배치], padding=True, truncation=True,
                      max_length=64, return_tensors="pt")
    입력["labels"] = torch.tensor([y for _, y in 배치])
    return 입력
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=3e-5, weight_decay=0.01)
스케줄러 = get_linear_schedule_with_warmup(옵티마이저, int(0.1 * 전체_걸음), 전체_걸음)
    for 배치 in 학습_로더:                              # 에폭 반복 안쪽
        출력 = 모델(**배치)                            # labels를 함께 주면 손실까지 계산해 준다
        출력.loss.backward()
        torch.nn.utils.clip_grad_norm_(모델.parameters(), 1.0)
```

lr 3e-5: 이미 좋은 자리, 조금만 옮긴다

워밍업 10%: 새 머리의 엉뚱한 기울기로부터 보호

5천 개 · 2에폭, 시험 5천 개, GPU

13.9 30분의 1의 데이터로 기준을 넘다

```
미세조정 전 시험 정확도: 0.4182
에폭 1: 학습 손실 0.4201, 시험 정확도 0.8518
에폭 2: 학습 손실 0.2249, 시험 정확도 0.8620
```



라벨 5천 개와 15만 개는 **전혀 다른 일**. 미세조정 전 41.82%는 머리가 무작위라서

14차시 · 13.9 미세조정하기 · 예제 13-korean-nlp/b22_finetune.py 출력 · 막대 길이 = 시험 정확도(0~100%), 시험 리뷰 5천 개

13.10 혼동 행렬과 F1

혼동 행렬 (행 = 정답, 열 = 예측)

	부정 예측	긍정 예측
정답 부정	**2,192**	346
정답 긍정	344	**2,118**

두 방향의 실수가 거의 같다: 346 대 344

classification_report (시험 리뷰 5,000개)

	정밀도	재현율	F1	개수
부정	0.864	0.864	0.864	2538
긍정	0.860	0.860	0.860	2462
정확도			**0.862**	5000

균형 잡힌 데이터라 정확도만 봐도 크게 속지 않는 경우. 한쪽이 드문 문제라면 이 표가 훨씬 중요하다

13.10 가장 자신 있게 틀린 리뷰를 읽는다

```
3475번: 정답 0 / 예측 1 (0.995), 13글자
345번: 정답 0 / 예측 1 (0.995), 12글자
668번: 정답 0 / 예측 1 (0.994), 10글자
853번: 정답 0 / 예측 1 (0.994), 19글자
2666번: 정답 0 / 예측 1 (0.994), 10글자
틀린 리뷰 690개 중 확률 0.95 이상으로
틀린 것: 145개
```

책의 실행에서 다섯 개를 읽어 보니

셋: 누가 읽어도 칭찬인데 라벨이 부정

하나: 말줄임표 붙은 칭찬, 사람도 갈림

하나: 지루하다는 불평, 모델이 틀림

모델의 약점뿐 아니라 **데이터의 약점**이 보인다. 시험 라벨이 틀렸다면 86.2%라는 숫자에도 그만큼 오차가 있다

14차시 · 13.10 평가: 숫자 너머를 보기 · 예제 13-korean-nlp/b23_evaluate.py (출력 뒷부분)

13.10 pipeline으로 바로 써 보기

```
분류기 = pipeline("text-classification", model="data/감성모델")
for 예문, 결과 in zip(예문들, 분류기(예문들)):
    print(f"{결과['label']} {결과['score']:.3f}  {예문}")
```

```
긍정 0.994  배우들 연기가 정말 좋았어요
부정 0.972  표값이 아까운 두 시간이었다
긍정 0.882  재미없지는 않았다    # 이중 부정
긍정 0.673  와 정말 대단하다 두 시간을 이렇게 버리게 해 주다니    # 비꼬기
긍정 0.983  ㄹㅇ 개꿀잼 인정    # 줄임말
```

이중 부정을 읽었다: 순서를 보지 않는 평균 임베딩은 약한 문장

비꼬기는 틀렸다. 쓸 곳에 들어올 문장을 시험 데이터에 일부러 넣어 점검한다

13.11 Hugging Face datasets로 다루기

```
자료 = load_dataset("csv", data_files={"train": "data/nsmc/ratings_train.txt",
                                       "test": "data/nsmc/ratings_test.txt"}, delimiter="\t")
자료 = 자료.filter(lambda 줄: 줄["document"] is not None and 줄["document"].strip() != "")
작은_학습 = 자료["train"].shuffle(seed=0).select(range(5000))
토큰화 = 작은_학습.map(lambda 줄: 토크나이저(줄["document"], truncation=True, max_length=64),
                       batched=True, remove_columns=["id", "document"])
로더 = DataLoader(토큰화, batch_size=4, collate_fn=DataCollatorWithPadding(토크나이저))
```

```
빈 리뷰를 걸러 낸 뒤: 149995 49997
{'labels': (4,), 'input_ids': (4, 64), 'token_type_ids': (4, 64), 'attention_mask': (4, 64)}
문장별 실제 토큰 수: [18, 64, 16, 15]
```

map(batched=True): 1,000줄씩 한꺼번에

동적 패딩: 묶을 때 배치 안 최대에 맞춘다

디스크 캐시: 메모리보다 큰 데이터도

14차시 · 13.11 Hugging Face datasets로 데이터 다루기 · 예제 13-korean-nlp/b25_datasets.py (핵심 부분, 출력 일부)

직접 실행 10분

1. b24를 복사해 예문을 바꾼다. 먼저 긍정·부정을 예상하고 실행한다
2. b21을 복사해 자르는 길이 목록에 48을 넣는다. 잘리는 리뷰는 몇 %일까?
3. b23 끝에 print(시험[i][0])을 넣어 가장 자신 있게 틀린 다섯 리뷰를 읽고 "라벨이 틀림 / 애매 / 모델이 틀림"으로 나눈다

```
예문들 = ["기대 안 했는데 의외로 괜찮네", "배우는 좋은데 각본이 다 망쳤다",
          "두 번 보세요 세 번 보세요", "이걸 돈 주고 봤다니"]        # 1번
for 자르기 in [32, 48, 64]:                                          # 2번
```

14차시 · 직접 실행 · 1번·2번 6초, 3번 29초 (b22가 data/감성모델을 저장한 뒤에)

미세조정의 학습률을 3e-5처럼 작게 잡는 이유는?

Q

가 GPU 메모리를 아끼려고

나 사전학습으로 배운 것을 잊지 않게 조금만 옮기려고

다 분류 머리를 학습하지 않으려고

라 에폭 수를 늘리려고

14차시 정리

쓰기 전에 **라이선스**와 크기부터. KLUE BERT는 CC BY-SA 4.0, float32 422MB, 학습엔 4배

미세조정: 작은 학습률, 워밍업, 클리핑. 리뷰 5천 개로 86.20%, 기준 모델 15만 개의 85.00%를 넘었다

혼동 행렬·F1에 더해 **자신 있게 틀린 예제**를 읽는다. pipeline으로 추론, datasets로 손질

다음 차시: 실습 · 같은 방법으로 뉴스 제목 주제 분류기

15차시

15

실습: 뉴스 제목 주제 분류기

책 13.12 ~ 13.13절

학습 목표

- Hub의 데이터셋을 받아 일곱 주제 분류기를 미세조정한다
- 주제별 F1과 두 가지 평균(macro, weighted)을 읽는다
- 13장 전체를 한 흐름으로 정리한다

지금 실행: **uv run python b26_lab.py** · GPU 98초

13.12 KLUE YNAT: 기사 제목의 일곱 주제

IT과학 · 경제 · 사회 · 생활문화 · 세계 · 스포츠 · 정치 | 학습 4만 5천여, 검증 9천여 | CC BY-SA 4.0

1

datasets로 받아 학습 6,000 · 검증 3,000 고르기

2

KLUE BERT 토크나이저, 48토큰에서 자르기

3

분류 머리를 주제 7개짜리로

4

14차시의 루프로 두 에폭

5

주제별 F1, 새 제목 넣어 보기

바뀌는 것은 데이터와 머리뿐. 인코더도, 루프도 감성 분류 그대로

13.12 1~2단계: 데이터와 토큰화

```
자료 = load_dataset("klue/klue", "ynat")
주제들 = 자료["train"].features["label"].names
학습 = 자료["train"].shuffle(seed=0).select(range(6000))
검증 = 자료["validation"].shuffle(seed=0).select(range(3000))

def 토큰화(묶음: dict) -> dict:
    return 토크나이저(묶음["title"], truncation=True, max_length=48)

열 = ["guid", "title", "url", "date"]
학습 = 학습.map(토큰화, batched=True, remove_columns=열).rename_column("label", "labels")
```

```
주제: ['IT과학', '경제', '사회', '생활문화', '세계', '스포츠', '정치']
트럼프 앙숙 워런 트럼프 2020년엔 자유인 아닐 수도 → 세계
```

13.12 3~4단계: 머리만 7개짜리로 바꿔 학습

```
모델 = AutoModelForSequenceClassification.from_pretrained(
    이름, num_labels=len(주제들), id2label=dict(enumerate(주제들)),
    label2id={t: i for i, t in enumerate(주제들)}).to(장치)

에폭 = 2
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=5e-5, weight_decay=0.01)
스케줄러 = get_linear_schedule_with_warmup(옵티마이저, int(0.1 * 전체_걸음), 전체_걸음)
```

```
에폭 1: 학습 손실 0.7238
에폭 2: 학습 손실 0.2734
```

분류 머리 = **nn.Linear(768, 7)**. lr 5e-5는 흔히 쓰는 범위(2e-5~5e-5)의 위쪽

13.12 5단계: 주제별 F1 · 검증 정확도 85.7%



막대 길이 = F1 (0~1, 높은 순)

찍으면 14% 정도인 문제에서 85.7%

"반도체", "스마트폰"은 경제에도 IT에도. 가장 어려운 두 주제

주제별로 보면 **어디에 데이터를 더 모을지** 보인다

13.12 검증 데이터는 고르지 않다: 두 가지 평균

주제	검증 개수 (support)
사회	**1,186**
경제	461
생활문화	451
세계	264
정치	241
스포츠	208
IT과학	189

macro avg F1

0.861

주제별 점수를 그냥 평균

weighted avg F1

0.857

개수로 가중한 평균

실제로 들어올 데이터의 분포를 생각하고 **어느 평균을 볼지** 정한다

13.12 새 제목 넣어 보기

```
입력 = 토크나이저(제목, return_tensors="pt").to(장치)
with torch.no_grad():
    확률 = 모델(**입력).logits.softmax(-1)[0]
print(f"{제목} → {주제들[확률.argmax()]} ({확률.max():.2f})")
```

```
반도체 수출 석 달 연속 증가 → 경제 (0.99)
국가대표팀 월드컵 예선 첫 승 → 스포츠 (0.98)
주말 전국에 비 소식 기온 뚝 → 생활문화 (0.99)
새 스마트폰 인공지능 비서 탑재 → IT과학 (0.98)
```

날씨가 생활문화인 건 이 데이터가 날씨 기사를 그렇게 묶기 때문. 모델은 우리 상식이 아니라 **데이터를 만든 사람의 기준**을 배운다

15차시 · 13.12 실습 6) 새 제목 · 예제 13-korean-nlp/b26_lab.py (끝부분, 제목은 예제용으로 지어낸 것)

직접 실행 10분

1. b26을 복사해 학습 데이터를 6,000개에서 **2,000개**로 줄인다. 검증 정확도 85.7%와 IT과학 F1 0.806은 어떻게 될까? 먼저 예상한다
2. 같은 파일의 6단계에 내가 지은 제목 두 개를 더 넣어 본다. 날씨처럼 기준이 갈릴 만한 제목을 골라 본다

```
학습 = 자료["train"].shuffle(seed=0).select(range(2000))      # 실습: 6000 → 2000
```

GPU(MPS): 원래 판 **98초**, 2,000개 판 **55초**

CPU만 있으면 2,000개 판 **86초**: 이 판으로 실습한다

감성 분류기를 주제 분류기로 바꿀 때 바꾸지 않아도 되는 것은?

Q

가 num_labels와 id2label

나 학습·검증 데이터

다 KLUE BERT 토크나이저와 인코더

라 분류 머리 nn.Linear의 출력 크기

13.13 13장 한눈에

한국어의 어려움

교착어, 들쭉날쭉한 띄어쓰기, NFD. 처리 전에 NFC

토큰화의 줄다리기

글자·어절·형태소·서브워드, 사전 크기 대 문장 길이

사전학습 토크나이저

모델과 한 몸. input_ids, attention_mask, 특수 토큰

임베딩

번호로 줄을 꺼내는 표, 코사인 유사도, 문맥 임베딩

기준 모델과 미세조정

5천 개 74.2% · 15만 개 85.0% · KLUE BERT 5천 개 86.2%

평가와 습관

자신 있게 틀린 예제 읽기, datasets, pipeline, 라이선스 확인

15차시 정리

Hub의 데이터셋을 이름으로 받고, **머리만 갈아 끼워** 일곱 주제 분류기를 만들었다: 검증 85.7%

주제별 F1로 약한 곳(IT과학, 경제)을 찾고, 분포를 보고 **macro와 weighted** 중 고른다

모델은 우리 상식이 아니라 **데이터를 만든 사람의 기준**을 배운다

다음 차시(4일차 16차시): 읽는 모델에서 쓰는 모델로 · 사전학습 언어 모델과 다음 토큰 예측

3일차에 배운 것

1. 한국어는 어절이 폭발한다: 리뷰 15만에 **35만 어절**, OOV 25.2%
2. 형태소와 **서브워드**로 사전 크기와 문장 길이의 중간을 찾는다
3. **임베딩**은 쓰이는 자리가 비슷한 말을 모으고, 인코더가 문맥을 더한다
4. KLUE BERT **미세조정**: 5천 개로 86.20%, 틀린 예제에서 라벨 잡음을 본다
5. 머리만 바꿔 **뉴스 주제 분류기** 85.7%

내일 4일차 (14장)

- 사전학습 언어 모델과 다음 토큰 예측
- 디코딩, 프롬프트와 채팅 템플릿
- 메모리 계산과 양자화
- LoRA와 peft, 1권 도우미 미세조정 실습